CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 1

Name: Prionti Das
UID: 24BAI71005
Branch: CSE-AIML(CS221)
Section/Group: 24AIT_NTPP-3(B)
Semester: 5th
Date of Performance: 08/07/2026
Subject: Full Stack-II
Subject Code: 24CSP-337

EXPERIMENT – 1.1

1. AIM:

To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

2. OBJECTIVE:

1. To understand multi-platform content handling
2. To implement real-time validation mechanisms
3. To design responsive and user-friendly UI components

3. SOFTWARE REQUIREMENTS:

1. Windows
1. Node.js (LTS Version)
2. npm (Node Package Manager)
3. Visual Studio Code
4. React.js

4. THEORY:

React is an open-source JavaScript library developed by Meta (Facebook) for building fast, dynamic, and interactive user interfaces. It follows a component-based architecture, where the user interface is divided into reusable components. This approach improves code reusability, readability, and maintainability. In modern social media applications, users often create content for multiple platforms such as Twitter, LinkedIn, and Instagram. Each platform has different character limits and posting rules. Therefore, a Post Composer is designed to help users create content while validating platform-specific constraints before publishing. Real-time validation improves user experience by immediately informing users if the entered content exceeds the allowed limit. The Post Composer developed in this experiment is implemented using React Functional Components and React Hooks. The useState() hook is used to store and manage dynamic data such as the selected platform and the content entered by the user. Whenever the state changes, React automatically updates the user interface without reloading the entire webpage. User interactions such as typing in the textarea or selecting a platform are handled using event handling (onChange). The application also uses controlled components, where the value of form elements is managed through React state. This ensures synchronization between the user interface and the application's data. The application dynamically calculates the number of characters entered using the length property and compares it with the predefined character limit of the selected platform. Based on this comparison, conditional rendering is used to display appropriate validation messages such as "Ready to Post" or "Character Limit Exceeded." Overall, this experiment demonstrates the practical implementation of React concepts including components, JSX, state management, event handling, controlled components, dynamic rendering, and real-time validation, providing the foundation for building modern frontend applications.

5. IMPLEMENTATION/PROCEDURE:

1. Install Node.js and verify the installation using node -v and npm -v.
2. Create a React project using npx create-react-app post-composer.
3. Open the project in Visual Studio Code and run it using npm start.
4. Design the Post Composer interface with a heading, platform dropdown, and textarea.
5. Implement React State (useState) to manage the selected platform and post content.
6. Add a live character counter using the length property.
7. Define platform-specific character limits (Twitter, LinkedIn, and Instagram).
8. Implement real-time validation to display appropriate messages based on the selected platform's character limit.
9. Test the application by entering different post lengths and switching between platforms.
10. Verify that the application displays the correct character count and validation message for each platform.

6. CODE :

```
import { useState } from "react";
 function App() {
 const [platform, setPlatform] = useState("Twitter");
 const [post, setPost] = useState("");

 const limits = {
  Twitter: 280,
  LinkedIn: 3000,
  Instagram: 2200,
 };

 const limit = limits[platform];
 const isExceeded = post.length > limit;

 return (
  <div
   style={{
    backgroundColor: "#f4f6f9",
    minHeight: "100vh",
    padding: "40px",
    fontFamily: "Arial, sans-serif",
   }}
  >
   <div
    style={{
     width: "650px",
     margin: "auto",
     backgroundColor: "#ffffff",
     padding: "30px",
     borderRadius: "15px",
     boxShadow: "0px 4px 15px rgba(0,0,0,0.2)",
    }}
   >
    <h1
     style={{
      textAlign: "center",
      color: "#2c3e50",
      marginBottom: "30px",
     }}
    >
```

```
  Social Media Post Composer
</h1>

<label>
  <b>Select Platform</b>
</label>

<br />
<br />

<select
  value={platform}
  onChange={(e) => setPlatform(e.target.value)}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "8px",
    fontSize: "16px",
  }}
>
  <option>Twitter</option>
  <option>LinkedIn</option>
  <option>Instagram</option>
</select>

<br />
<br />

<label>
  <b>Write Your Post</b>
</label>

<br />
<br />

<textarea
  rows="6"
  placeholder="Type your post here..."
  value={post}
  onChange={(e) => setPost(e.target.value)}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "10px",
    border: "2px solid #ccc",
    fontSize: "16px",
    resize: "none",
  }}
></textarea>

<br />
<br />

<p
```

```
          style={{
            fontWeight: "bold",
            color: isExceeded ? "red" : "#2c3e50",
          }}
        >
          Characters: {post.length} / {limit}
        </p>

        {isExceeded ? (
          <p
            style={{
              color: "red",
              fontWeight: "bold",
            }}
          >
            Character Limit Exceeded!
          </p>
        ) : (
          <p
            style={{
              color: "green",
              fontWeight: "bold",
            }}
          >
            Ready to Post
          </p>
        )}

        <button
          style={{
            backgroundColor: "#007bff",
            color: "white",
            border: "none",
            padding: "12px 20px",
            borderRadius: "8px",
            fontSize: "16px",
            cursor: "pointer",
            marginTop: "10px",
          }}
        >
          Save Draft
        </button>
      </div>
    </div>
  );
}

export default App;
```

7. OUTPUT :



Fig 1.1: Social Media Post Composer User Interface

The application displays a **Social Media Post Composer** interface where the user can select a social media platform (Twitter, LinkedIn, or Instagram), type a post, and view the live character count. It validates the post against the selected platform's character limit and displays **"Ready to Post"** if the limit is not exceeded. A **Save Draft** button is also provided to save the composed post.

8. LEARNING OUTCOME:

- Understand the fundamentals of React.js and component-based development.
- Create and run a React application using Node.js and npm.
- Design a dynamic Post Composer interface using React.
- Implement state management using the useState() Hook.
- Handle user input using controlled components and event handling.
- Develop interactive and responsive frontend applications using React.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 1.2

1. AIM:

To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.

2. OBJECTIVE:

1. To manage draft data using frontend state
2. To implement CRUD operations for draft handling
3. To understand asynchronous workflows in UI applications
4. To optionally simulate backend communication using mock APIs

3. SOFTWARE REQUIREMENTS:

1. Windows
2. Node.js (LTS Version)
3. npm (Node Package Manager)
4. Visual Studio Code
5. Google Chrome
6. React.js
7. Command Prompt

4. THEORY:

A Draft Management System enables users to temporarily save content before publishing it. It improves user experience by allowing posts to be edited, updated, or deleted at any time. In this experiment, React's useState() Hook is used to manage draft data, while useEffect() and Local Storage are used to persist drafts even after refreshing the browser. The application also demonstrates CRUD operations, dynamic rendering using map(), and array manipulation using filter(), providing practical knowledge of frontend state management.

5. IMPLEMENTATION/PROCEDURE:

1. Create a React State to store multiple drafts.
2. Implement the Save Draft functionality using useState().
3. Display all saved drafts using the map() function.
4. Implement Edit Draft functionality.
5. Implement Delete Draft functionality using filter().
6. Store drafts in the browser using Local Storage.
7. Load saved drafts automatically using useEffect().
8. Test all CRUD operations.

6. CODE:

```
import { useState } from "react";

function App() {
  const [platform, setPlatform] = useState("Twitter");
  const [post, setPost] = useState("");
  const [drafts, setDrafts] = useState([]);
```

```
const limits = {
  Twitter: 280,
  LinkedIn: 3000,
  Instagram: 2200,
};

const limit = limits[platform];
const isExceeded = post.length > limit;

const saveDraft = () => {
  if (post.trim() === "") {
    alert("Please write something first!");
    return;
  }

  setDrafts([...drafts, post]);
  setPost("");
};

const editDraft = (index) => {
  setPost(drafts[index]);

  const updatedDrafts = drafts.filter((_, i) => i !== index);
  setDrafts(updatedDrafts);
};

const deleteDraft = (index) => {
  const updatedDrafts = drafts.filter((_, i) => i !== index);
  setDrafts(updatedDrafts);
};

return (
  <div
    style={{
      maxWidth: "700px",
      margin: "40px auto",
      padding: "20px",
      fontFamily: "Arial",
      backgroundColor: "#f8f9fa",
      borderRadius: "12px",
      boxShadow: "0 0 10px rgba(0,0,0,0.2)",
    }}
  >
    <h1 style={{ textAlign: "center" }}>
      Social Media Post Composer
    </h1>

    <h3>Select Platform</h3>

    <select
      value={platform}
      onChange={(e) => setPlatform(e.target.value)}
      style={{
        width: "100%",
```

```
    padding: "10px",
    borderRadius: "8px",
    fontSize: "16px",
    marginBottom: "20px",
   }}
  >
   <option value="Twitter">Twitter</option>
   <option value="LinkedIn">LinkedIn</option>
   <option value="Instagram">Instagram</option>
  </select>

  <h3>Write Your Post</h3>

  <textarea
   value={post}
   onChange={(e) => setPost(e.target.value)}
   rows="6"
   placeholder="Write your post here..."
   style={{
    width: "100%",
    padding: "10px",
    borderRadius: "10px",
    border: "2px solid #ccc",
    fontSize: "16px",
    resize: "none",
   }}
  />

  <p>
   Characters: {post.length} / {limit}
  </p>

  {isExceeded ? (
   <p style={{ color: "red", fontWeight: "bold" }}>
    Character limit exceeded!
   </p>
  ) : (
   <p style={{ color: "green", fontWeight: "bold" }}>
    Ready to post
   </p>
  )}

  <button
   onClick={saveDraft}
   disabled={isExceeded}
   style={{
    backgroundColor: "#007bff",
    color: "white",
    border: "none",
    padding: "10px 20px",
    borderRadius: "8px",
    cursor: isExceeded ? "not-allowed" : "pointer",
    fontSize: "16px",
   }}
```

```
>
  Save Draft
</button>

<hr style={{ margin: "30px 0" }} />

<h2>Saved Drafts</h2>

{drafts.length === 0 ? (
  <p>No drafts available.</p>
) : (
  drafts.map((draft, index) => (
    <div
      key={index}
      style={{
        backgroundColor: "#fff",
        padding: "15px",
        borderRadius: "10px",
        marginBottom: "15px",
        boxShadow: "0 0 5px rgba(0,0,0,0.1)",
      }}
    >
      <p>{draft}</p>

      <button
        onClick={() => editDraft(index)}
        style={{
          backgroundColor: "#28a745",
          color: "white",
          border: "none",
          padding: "8px 15px",
          borderRadius: "5px",
          cursor: "pointer",
          marginRight: "10px",
        }}
      >
        ✏ Edit
      </button>

      <button
        onClick={() => deleteDraft(index)}
        style={{
          backgroundColor: "#dc3545",
          color: "white",
          border: "none",
          padding: "8px 15px",
          borderRadius: "5px",
          cursor: "pointer",
        }}
      >
        🗑 Delete
      </button>
    </div>
  ))
```

```
      )}
    </div>
  );
}

export default App;
```

7. OUTPUT :

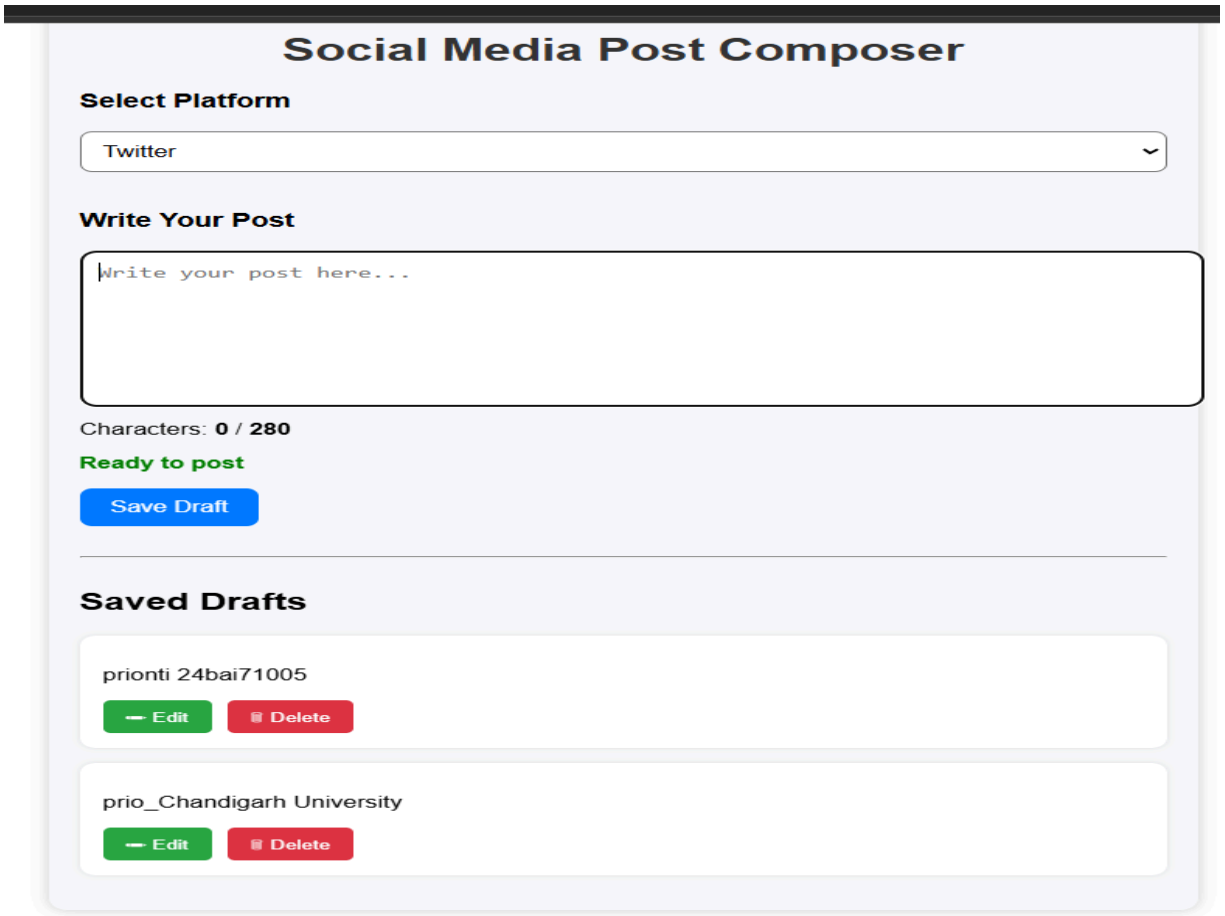


Fig 1.2: Draft Management System Webpage

The output displays a **Social Media Post Composer** built using React. Users can select a social media platform (Twitter, LinkedIn, or Instagram), write a post, and view a live character count based on the selected platform's limit. The application validates the character limit and displays the status as **"Ready to post"** or **"Character limit exceeded."** Users can save posts as drafts, and the saved drafts are displayed below with **Edit** and **Delete** options for easy management. The interface is simple, responsive, and user-friendly, demonstrating the use of React state management and event handling.

8. LEARNING OUTCOME:

- Understand the concept of Draft Management.
- Implement CRUD operations in React.
- Manage application data using React State.
- Display dynamic data using the map() function.
- Modify arrays using the filter() function.
- Use the useEffect() Hook for lifecycle management.
- Store and retrieve data using Local Storage.